

Universidad Rafael Landívar
Facultad de Ingeniería · Base de Datos II

EduGT

Academia en línea

Documentación Técnica — Fase 2

Flujo de registro y publicación de cursos

Justificación de aislamiento · Bloqueos · Referencia T-SQL

Integrantes

Sofía Paiz · 1092224

Gabriel Ajín · 1184924

Daniel Xiquín · 1118423

Ricardo De la Cruz · 1176323

2026

1. Introducción

Este documento describe la solución T-SQL de la Fase 2 del proyecto EduGT, correspondiente al flujo de registro y publicación de cursos. La solución se implementa íntegramente en la capa de base de datos mediante procedimientos almacenados, sin lógica de aplicación externa, de modo que todas las reglas de negocio y las garantías de integridad se hacen cumplir dentro del motor.

El flujo cubre la creación de un curso por parte de un instructor, su revisión por el comité académico, y su publicación automática en el catálogo al ser aprobado. Se documentan además las decisiones de concurrencia, aislamiento y bloqueo tomadas en cada punto de contención, y se presenta la referencia técnica de cada procedimiento.

2. Arquitectura del flujo

El ciclo de vida de un curso atraviesa los siguientes estados, controlados por la columna `courses.status`:

- **pending**: estado inicial al crearse el curso; espera revisión.
- **under_review**: un académico tomó el curso para revisarlo.
- **available**: aprobado y publicado en el catálogo; visible para inscripción.
- **rejected**: rechazado con comentarios; el instructor puede corregir y reenviar.

Los procedimientos que gobiernan estas transiciones son: `sp_CreateCourse`, `sp_ClaimCourseForReview`, `sp_ApproveCourse`, `sp_RejectCourse` y `sp_ResubmitCourseForReview`. La configuración de la plataforma (comisiones, cohortes, política de reembolso y cursos destacados) se maneja con un grupo aparte de procedimientos ejecutados por el administrador.

3. Justificación de aislamiento y bloqueos

La solución identifica dos puntos de contención reales en esta fase y los resuelve dentro del motor, sin confiar en validaciones externas. A continuación se justifica cada mecanismo elegido.

3.1 Generación del código de curso — UPDLOCK + HOLDLOCK

El código de curso debe ser correlativo por año, único y sin huecos. El contador vive en la tabla `course_code_sequences` (una fila por año). El riesgo de concurrencia es doble: dos instructores creando cursos en el mismo instante podrían leer el mismo último número, o podrían intentar crear simultáneamente la fila del año nuevo.

La solución toma un bloqueo de actualización sobre la fila del año antes de incrementarla:

```
INSERT INTO course_code_sequences (year, last_number)
SELECT @year, 0
WHERE NOT EXISTS (
    SELECT 1 FROM course_code_sequences WITH (UPDLOCK, HOLDLOCK) WHERE year = @year);

UPDATE course_code_sequences WITH (UPDLOCK, ROWLOCK)
```

```
SET last_number = last_number + 1, @next_number = last_number + 1
WHERE year = @year;
```

El **UPDLOCK** evita que dos sesiones lean el mismo valor y produzcan códigos duplicados: la segunda sesión espera a que la primera libere el bloqueo. El **HOLDLOCK** (equivalente a aislamiento serializable sobre esa consulta) cierra la ventana entre la comprobación de existencia y la inserción de la fila del año nuevo, evitando que ambas sesiones intenten insertarla a la vez. Como el contador se incrementa mediante un UPDATE dentro de la misma transacción, si esta falla y hace ROLLBACK el número vuelve atrás, garantizando que no queden huecos permanentes; por eso no se usa una SEQUENCE nativa, que no revierte los números ya entregados.

3.2 Revisión única por curso — índice único filtrado

Un curso no puede tener dos revisiones activas al mismo tiempo: dos académicos no deben poder tomarlo en paralelo. Una validación previa dentro del procedimiento (un SELECT que comprueba si ya hay revisión activa) no basta, porque entre esa lectura y la inserción existe una ventana en la que ambas sesiones verían el curso libre.

La garantía se hace cumplir a nivel de motor con un índice único filtrado:

```
CREATE UNIQUE INDEX UX_reviews_active
ON academic_reviews (course_id) WHERE status = 'in_progress';
```

El índice permite muchas revisiones históricas (finished) del mismo curso, pero solo una in_progress. Cuando dos académicos ejecutan sp_ClaimCourseForReview simultáneamente, ambos intentan insertar una fila in_progress; el motor deja pasar solo la primera y rechaza la segunda con error 2601/2627. El procedimiento captura ese error y lo traduce a un mensaje de negocio claro: «Este curso ya está siendo revisado por otro académico». La atomicidad se resuelve en el INSERT mismo, no en una comprobación previa.

3.3 Resolución de revisiones — concurrencia optimista con @@ROWCOUNT

Al aprobar o rechazar, el procedimiento actualiza la revisión condicionando el UPDATE a que siga en progreso, y luego verifica cuántas filas cambió:

```
UPDATE academic_reviews SET result = 'approved', status = 'finished' ...
WHERE id = @review_id AND status = 'in_progress';
IF @@ROWCOUNT = 0 THROW 52002, 'La revision ya no esta en progreso...', 1;
```

Si otra sesión resolvió la misma revisión un instante antes, el UPDATE afecta cero filas y el procedimiento aborta con un mensaje claro en lugar de sobrescribir el resultado. Es control optimista: no se bloquea por adelantado, se detecta el conflicto en el momento de escribir.

3.4 Consistencia transaccional — XACT_ABORT y TRY/CATCH

Todos los procedimientos que escriben en más de una tabla usan SET XACT_ABORT ON junto con un bloque BEGIN TRY / BEGIN CATCH que revierte la transacción ante cualquier error y relanza la excepción. Esto garantiza que una operación como la creación de un curso (que inserta en courses, modules, lessons, course_instructors, course_prerequisites y notifications) sea todo-o-nada: nunca queda un curso a medio crear.

4. Referencia de procedimientos almacenados

4.1 sp_CreateCourse

Crea un curso completo con sus módulos, lecciones, co-instructores y prerrequisitos, en estado pending, y notifica al instructor. Valida toda la información antes de abrir la transacción.

Validaciones principales:

Error / Código	Número	Condición que lo dispara
Instructor inválido	50001	El instructor no existe o no tiene rol instructor.
Co-instructor inválido	50014	Algún co-instructor no existe o no tiene rol instructor.
Categoría inexistente	50002	La categoría indicada no existe.
Precio fuera de rango	50003	El precio está fuera del rango de platform_config.
Instructor principal	50004 / 50005	Debe haber exactamente un is_main = 1 y coincidir con el instructor.
Mínimo de módulos	50006	El curso debe tener al menos tres módulos.
Orden de módulos	50007	El order_index debe ser secuencial de 1 a N sin huecos.
Contenido por módulo	50008	Cada módulo debe tener al menos una lección con content_url.
Orden de lecciones	50009	El order_index de lecciones debe ser secuencial dentro de cada módulo.
Suma de participación	50010	La suma de share_percent debe ser exactamente 100.
Prerrequisitos	50011	Todos los prerrequisitos deben existir y estar available.
Tipo de lección	50016	El tipo debe ser video, document o quiz.

4.2 sp_ClaimCourseForReview

Permite a un académico tomar un curso pending o under_review para revisarlo. Inserta la revisión in_progress y cambia el curso a under_review. La unicidad de revisión activa la garantiza el índice filtrado (sección 3.2).

Error / Código	Número	Condición que lo dispara
Revisor inválido	51001	El revisor no existe o no tiene rol academic.
Estado del curso	51002	El curso no existe o no está en pending ni under_review.
Revisión duplicada	51003	El curso ya está siendo revisado por otro académico.

4.3 sp_ApproveCourse

Aprueba una revisión en progreso, publica el curso (status available, published_at) y notifica al instructor principal y a los suscriptores de la categoría. Cumple el requisito de publicación automática en el catálogo.

Error / Código	Número	Condición que lo dispara
Revisión inválida	52001	La revisión no existe, no está en progreso o no es de este revisor.
Curso no en revisión	52003	El curso asociado no está en estado under_review.

Error / Código	Número	Condición que lo dispara
Conflicto de concurrencia	52002	La revisión ya fue resuelta por otra sesión.

4.4 sp_RejectCourse

Rechaza una revisión en progreso con un comentario obligatorio, cambia el curso a rejected y notifica al instructor con el motivo. El comentario no puede estar vacío ni contener solo espacios.

Error / Código	Número	Condición que lo dispara
Comentario vacío	53001	Debe indicarse un comentario con el motivo del rechazo.
Revisión inválida	53002	La revisión no existe, no está en progreso o no es de este revisor.
Curso no en revisión	53004	El curso asociado no está en estado under_review.
Conflicto de concurrencia	53003	La revisión ya fue resuelta por otra sesión.

4.5 sp_ResubmitCourseForReview

Permite al instructor principal reenviar a revisión un curso rechazado, devolviéndolo a pending para un nuevo ciclo.

Error / Código	Número	Condición que lo dispara
Estado del curso	54001	El curso no existe o no está en estado rejected.
Autorización	54002	Solo el instructor principal puede reenviar el curso.
Conflicto de concurrencia	54003	El curso ya no está en estado rejected.

4.6 Procedimientos de configuración flexible

Ejecutados por el administrador (todos validan rol admin) para configurar la plataforma:

- **sp_SetCategoryCommission:** fija el porcentaje de comisión de una categoría.
- **sp_SetInstructorCommission:** registra una comisión preferencial por instructor con vigencia por fechas, cerrando la anterior.
- **sp_CreateCohort:** crea una cohorte con fecha de inicio, fin opcional y cupo máximo.
- **sp_SetRefundPolicy:** establece la política de reembolso vigente; inserta una nueva y desactiva la anterior para preservar el snapshot de las inscripciones existentes.
- **sp_SetCourseFeatured:** destaca o quita un curso de la portada.

5. Estrategia de pruebas

La solución incluye datos de prueba y verificaciones automáticas mediante procedimientos auxiliares (test_pass, test_fail, test_expect_error) que reportan PASS o FAIL en consola. Las pruebas se agrupan en:

- **Pruebas funcionales de sp_CreateCourse:** un caso válido y ocho casos de error, uno por cada validación clave.

- **Pruebas del flujo de revisión:** dos ciclos completos (aprobación y rechazo/reenvío) más cuatro casos de error.
- **Pruebas de concurrencia:** una prueba determinista que reproduce el estado de la carrera de revisión, más un guion para demostrar concurrencia real con dos sesiones simultáneas sincronizadas con WAITFOR TIME.

El punto de contención de sobreventa de cupos y actualización perdida de progreso corresponde a la Fase 3 (inscripción y avance), por lo que sus pruebas de concurrencia se abordarán en esa entrega. En la Fase 2 el único punto de contención es la unicidad de la revisión académica.